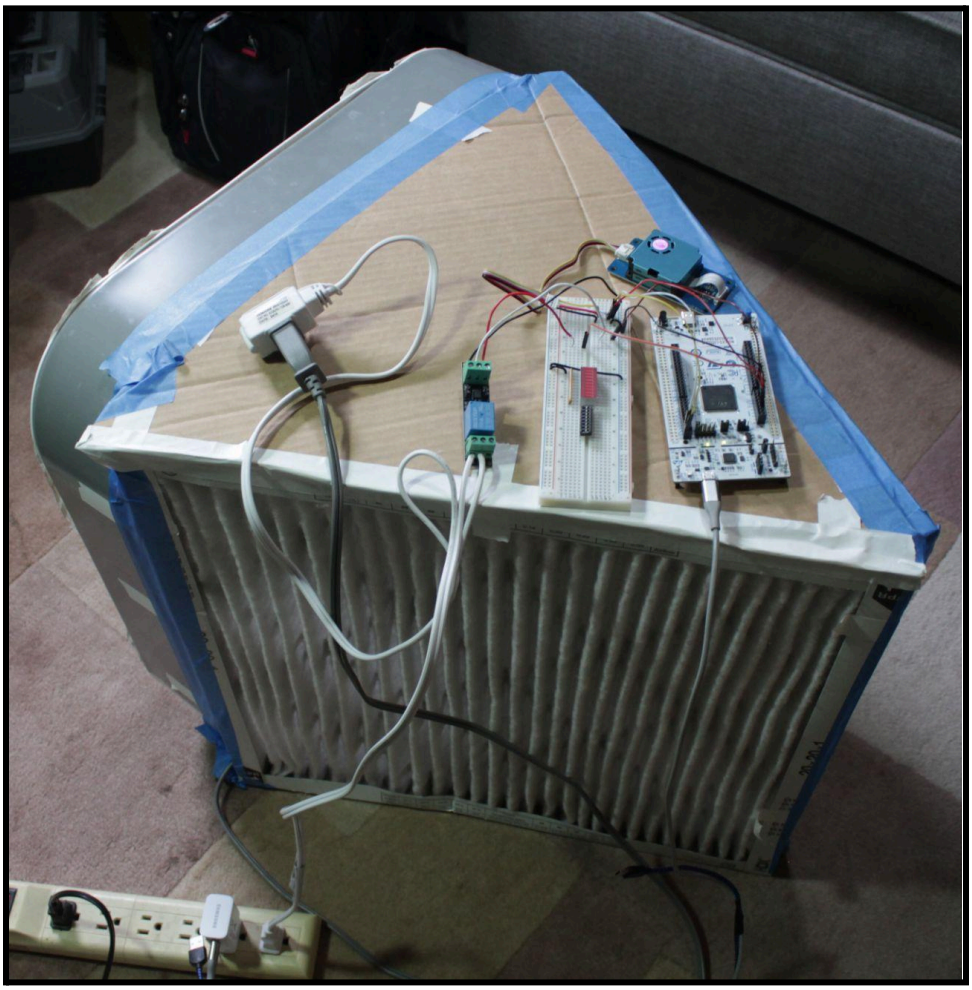
BOISE STATE UNIVERSITY

Spring 2025

ECE433/533/CS456



Automatic Air Analyzer

Product Design Specification

Designer: *Taylor Zaugg*

Version: *1.0*

05/01/2025

TABLE OF CONTENTS

1 Introduction **3**
1.1 Purpose of The Product 4
2 General Overview and Design Guidelines/Approach **4**
3 Architecture Design **4**
3.1 Logical View 4
3.2 Hardware Architecture 5
3.3 Software Architecture 6
3.4 Communication Architecture 7
3.5 Power considerations 7
3.6 Sensors/Actuators Description 7
4 System Design **10**
4.1 Bill of Material (BOM) 10
4.2 Calibration and test procedures 10
5 Conclusion on Product Design Specification **10**

1 Introduction

This product is designed to be an air purifier system. It will detect the air quality of a room and filter the air when it gets above a set threshold. There is an LED array indicator that provides visual information about how many particles are present in the air. The LEDs in the array will trigger based on a semi-logarithmic scale, where the higher LEDs require many more particles before lighting up than the lower LEDs. Since box fans can be quite loud, there is also a button to pause and start the system when desired.

1.1 Purpose of The Product

This product provides a solution to automatic air filtration needs. The product will detect particles in the air using a laser dust sensor, specifically the Seeed Studio's Grove - Laser PM2.5 Sensor (HM3301). If the particle count reaches higher levels than the defined threshold, the default threshold is based on the national household average particule matter (PM) concentration of 2.5 micron particles, the fan system will be powered on to filter the air [1]. By communicating with the HM3301 air sensor, the MCU can constantly monitor the air quality to know when to turn the air filtration system on.

2 General Overview and Design Guidelines/Approach

The product should have at least 12 inches of space on all sides for optimal airflow. Power should be connected to the STM32 via a micro USB connected to a 5V power brick or from a computer's USB port. The relay input should be connected to 120VAC, and the relay output should be connected to the box fan. Typical box fan models have a built-in speed control. This may still be manually adjusted based on the desired airflow when in operation. However, it is recommended to be set to the max for the quickest removal of particulate matter.

The design of this system is to provide a hands-free approach to controlling an air filtration system. Steps to achieve this revolved around developing firmware to read data from the air sensor and decide when to trigger the fan to start up. The goal is to have it consistently check the particle values and use them to determine when to trigger the filtration system. Box fans can be quite noisy, so a button should be available to pause and start the system whenever the user chooses. Automatic devices are also more friendly when there is a visual indication of what is happening. This is why the LED array is included in the design. This provides feedback on the current quality of air around the system.

3 Architecture Design

The HM3301 sensor sends data to the MCU about current air quality. This data is compared to threshold values initially determined by the national average of households. When the threshold is broken, the MCU signals the relay to provide power to a box fan. This box fan has two MERV-13 rated filters attached to the intake side. The fan forces air through the filters, trapping PM and cleaning the air.

3.1 Logical View

The block diagram in Figure 1 shows the high-level connection between the STM32 MCU and the components that make up the system. These other components are the HM3301

sensor, the fan control module, the fan, and the pushbutton affixed on the STM32 board itself.

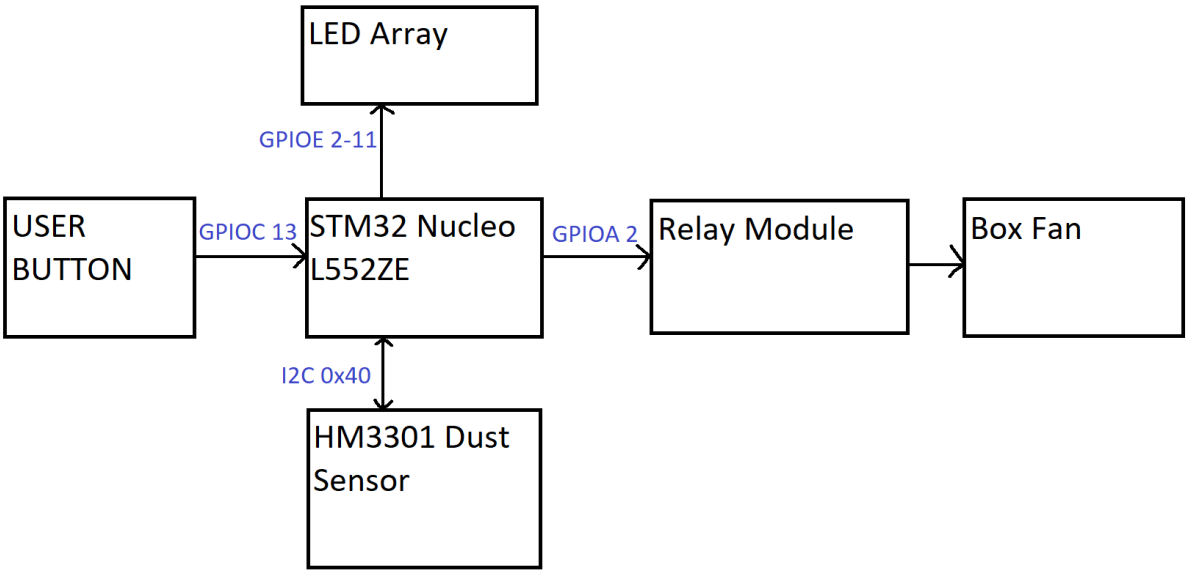


Figure 1 - Block diagram of the Automatic Air Analyzer system

3.2 Hardware Architecture

A wiring schematic showing how the components should be connected is in Figure 2. This figure shows the STM32 connecting to the LED array, labeled BAR1 in the figure, via GPIOE pins 2-11. This LED array is pulled to ground via a resistor network. Also shown is the HM3301 sensor and its I2C connections to the STM32 using GPIOB pins 8 and 9 in I2C mode. Wire routing between the relay module and the fan is similar to wiring of a common switch, where the hot wire is broken to remove power from components. The relay module is connected to the STM by tying its 3.3V_in line to 3.3V out from the STM32 and connecting the 3.3V_Signal to GPIOA pin 2.

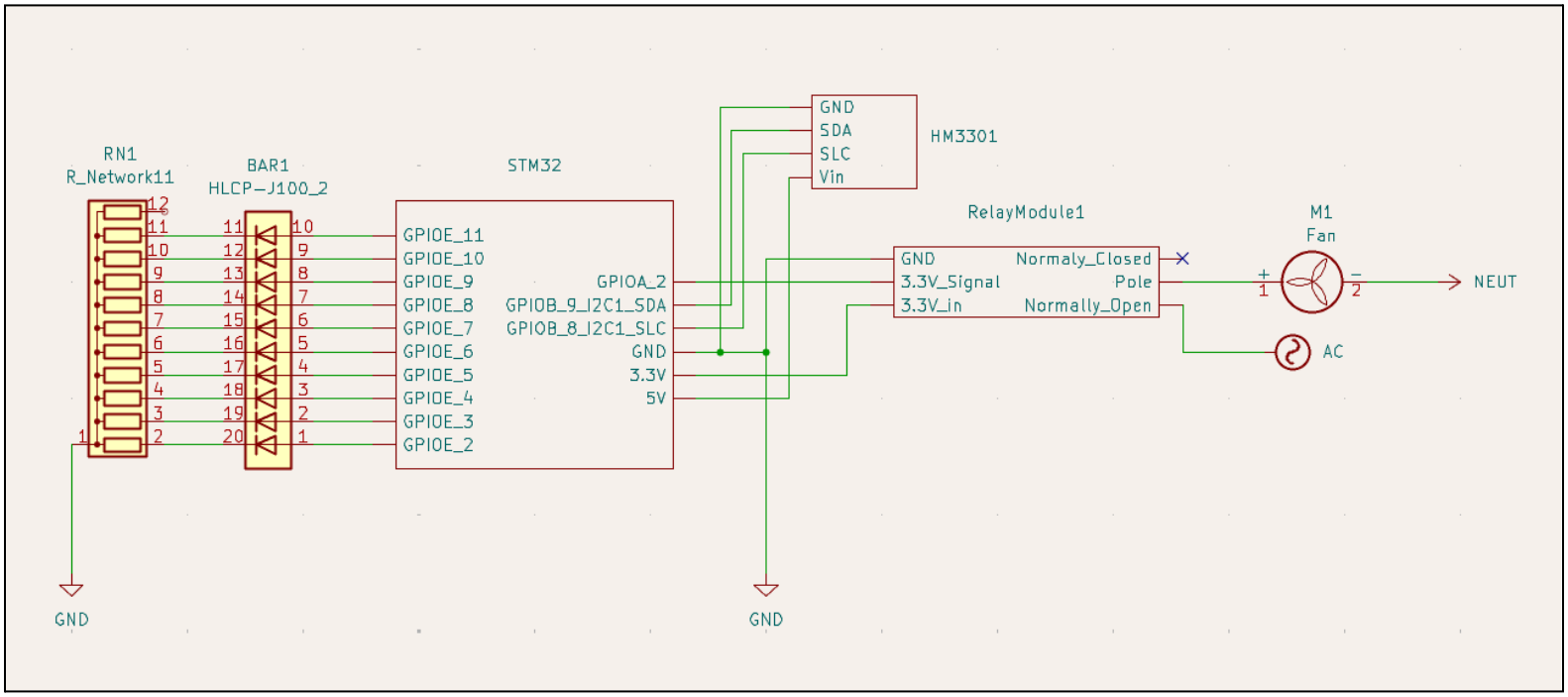


Figure 2. Schematic view of the Automatic Air Analyzer

3.3 Software Architecture

The two major sections in the software are the TIM2_IRQHandler, for the TIM2 interrupts, and the EXTI13_IRQHandler, for the user push button interrupts.

The TIM2_IRQHandler is where most of the program's time will be spent, outside of just waiting. TIM2 timer sends an interrupt every 5 seconds. This triggers the TIM2_IRQHandler's code, where the HM3301 sensor's data is read. This data is then used to set the LED array's output values. After this, the values are parsed for PM values 1, 2.5, and 10. These are compared with preprocessor-defined threshold values, which determine if the Relay Module, controlled by GPIOA pin 2, should be set to high or low.

At any time, the user can press the user push button. This will trigger an interrupt and move the program to the EXTI13_IRQHandler. In this function, the TIM2 timer will either be paused or restarted depending on the current state. Also, if the Relay Module is set high, then it will force it to turn off. Figure 3 shows a block diagram of the program's functionality from a higher level.

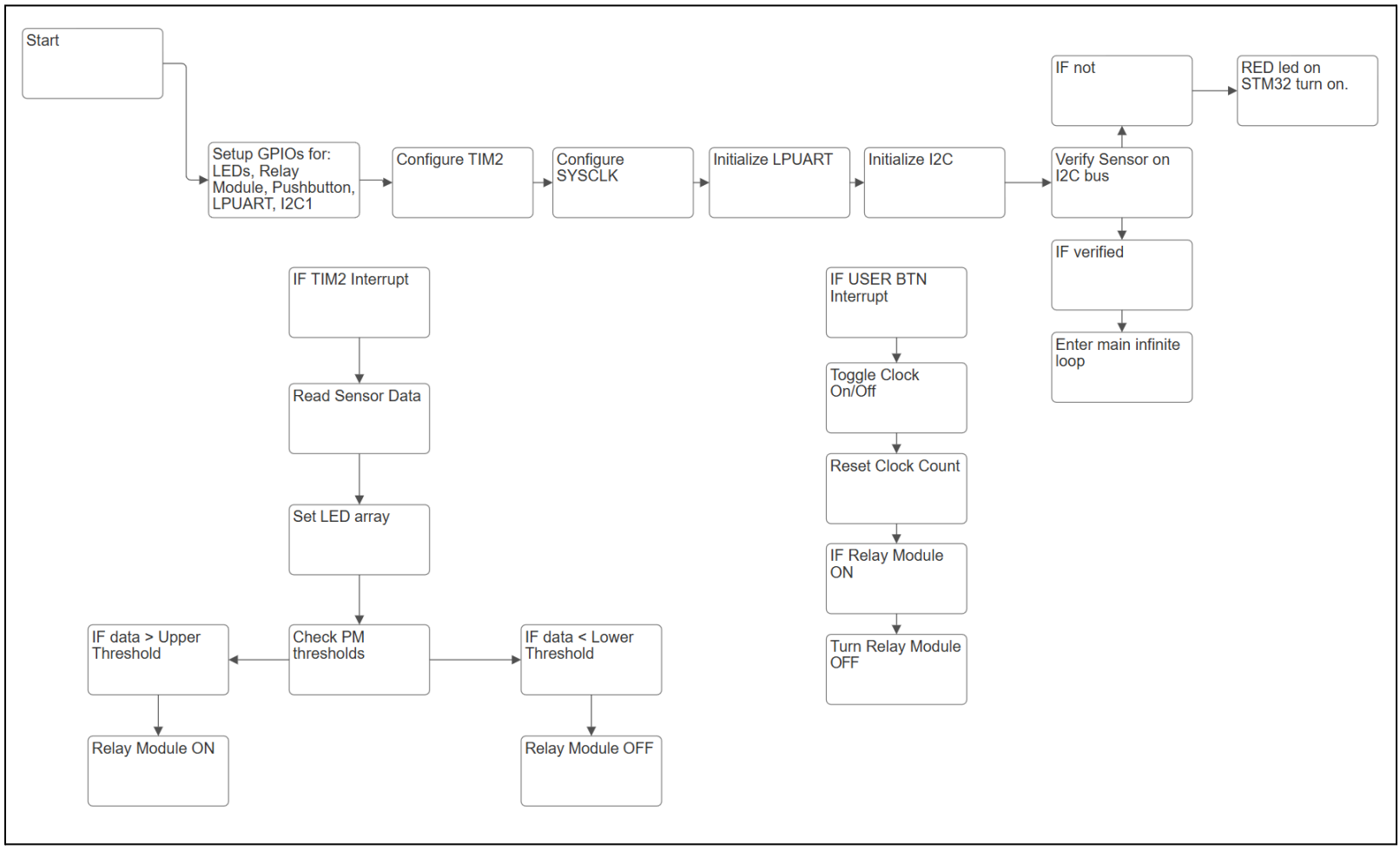


Figure 3. Software Diagram of the Automatic Air Analyzer

3.4 Communication Architecture

The main communication protocol used is I2C for the HM3301 sensor. The HM3301 datasheet provides information about how to communicate with it over I2C. The first command needed is the control instruction to put the sensor in I2C mode. Table 1 shows this control instruction as provided by the datasheet. It states the address of the device as 0x80, and the command for I2C mode is 0x88.

Table 1. Control instruction format with the UART disable command

Start	Address Write	Answer	Command	Answer	end
Start	0x80	ACK	0x88	ACK	Stop

The other information to know is how the sensor data will be sent back to the STM32. Table 2 shows the provided data frame format. The sensor will return all data from one read call.

Table 2. I2C Data Frame format

Start	Address Read	Answer	data	Answer	Data…	answer	end
Start	0x81	ACK	Data1	ACK	Data…n	ACK	Stop

Further in the datasheet, another table is provided showing what each data byte represents and how many bytes there are; this is shown in Table 3.

Table 3. Data format

Data1~2	Reserved
Data3~4	Sensor Number
Data5~6	PM1.0 concentration (CF = 1, Standard particulate) unit ug/m3
Data7~8	PM2.5 concentration (CF = 1, Standard particulate) unit ug/m3
Data9~10	PM10 concentration (CF = 1, Standard particulate) unit ug/m3
Data11~12	PM1.0 concentration (Atmospheric environment) unit ug/m3
Data13~14	PM2.5 concentration (Atmospheric environment) unit ug/m3
Data15~16	PM10 concentration (Atmospheric environment) unit ug/m3
Data17~18	the number of particles with diameter 0.3um or above in 1 liter of air
Data19~20	the number of particles with diameter 0.5um or above in 1 liter of air
Data21~22	the number of particles with diameter 1.0um or above in 1 liter of air
Data23~24	the number of particles with diameter 2.5um or above in 1 liter of air
Data25~26	the number of particles with diameter 5.0um or above in 1 liter of air
Data27~28	the number of particles with diameter 10um or above in 1 liter of air
Data29	Data0~Data28 Checksum

However, the address of 0x80 for the I2C address of the sensor is not correct. Seeed Studio attached another board to this sensor that only allows I2C mode and changes the address to 0x40. This is seen as printed on the board next to the 4-pin connection, and in the example code that Seeed Studio provides for Arduino boards. Despite this difference, the rest of the commands and data format are the same.

3.5 Power considerations

To power the STM32 microcontroller, a micro USB connection to a computer or a 5V power brick is required. The box fan relay requires a connection to a 120VAC house receptacle.

3.6 Sensors/Actuators Description

Seeed Studio - Grove - Laser PM2.5 Sensor (HM3301)

One of these sensors is used to provide the air quality information to the MCU via I2C. Figure 4 shows a photo of the component from DigiKey's website [3].

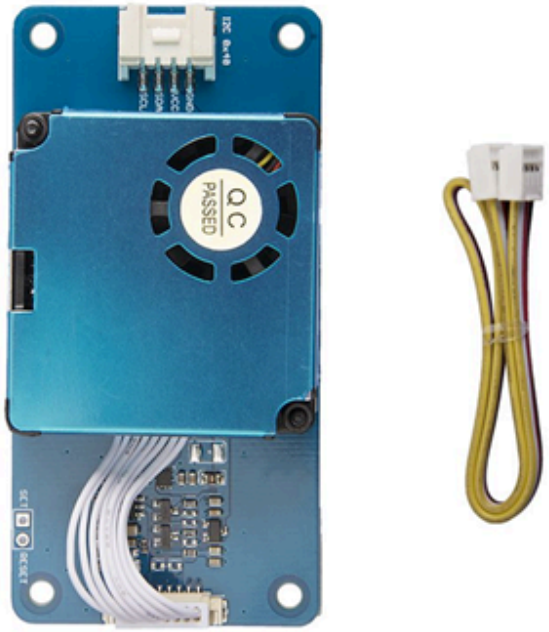


Figure 4. Image of Grove - Laser PM2.5 Sensor (HM3301)[3]

The explanation of how the sensor works is best left to the explanation provided by the datasheet, "The HM-3301 Dust Sensor is based on the advanced Mie scattering theory. When light passes through particles with quantity same as or larger than wavelength of the light, it will produce light scattering. The scattered light is concentrated to a highly sensitive photodiode, which is then amplified and analyzed by a circuit. With specific mathematical model and algorithm, the count concentration and mass concentration of the dust particles is obtained." [2]

Box Fan

The one box fan used provides the airflow to filter out particles from the air. Figure 5 shows a typical example of a box fan[4]. There are various brands, but really, this project

only needs the least expensive 20” box fan that can be found.

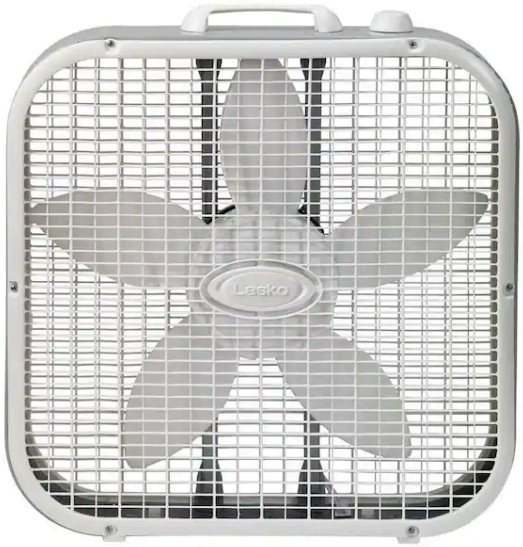


Figure 5. Box Fan example [4]

MERV-13 Filters

Two MERV-13 rated filters are taped to the box fan’s intake side to form a triangle from the top view. Using the cardboard from the box fan’s box, or other sourced cardboard, to create the top seal. Figure 6 shows a top down view of how the filters should be attached to the box fan.

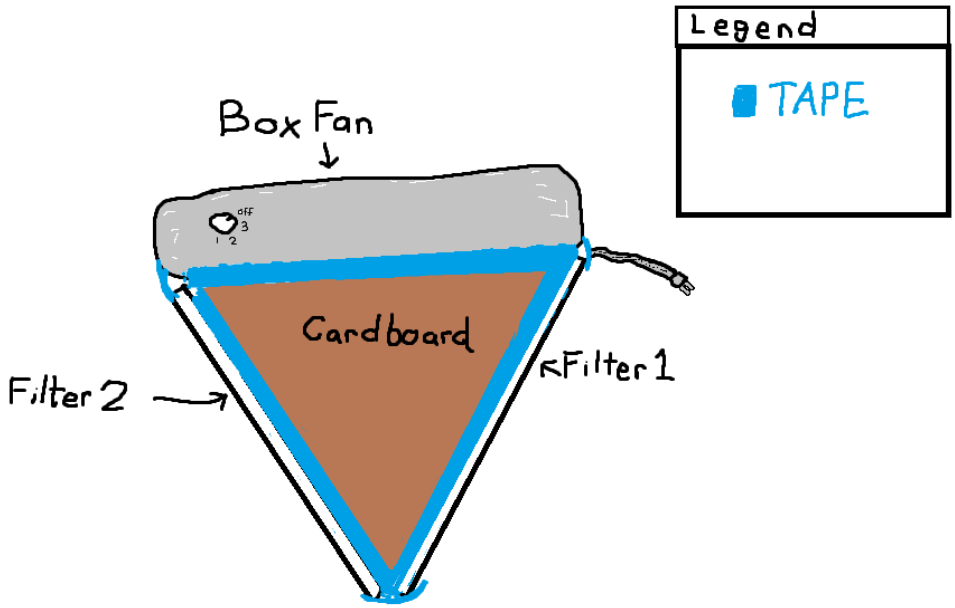


Figure 6. Top view of MERV-13 filter connection to the box fan

Relay Module

The relay module used allows a 3.3VDC signal to trigger a relay that switches our 120VAC signal. Figure 7 shows an image of the relay module used. Table 4 shows the connections it makes to other components.

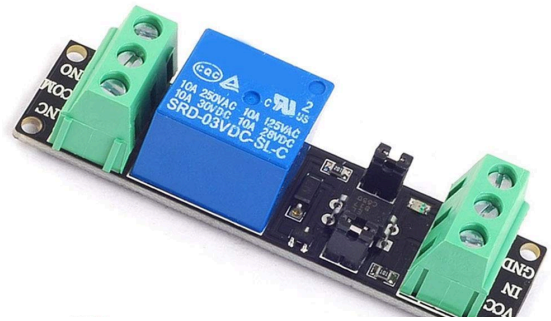


Figure 7. Relay Module [5]

Table 4. Relay to STM32 connection

Relay Module Pin	Connection
Vcc	STM32 - 3.3V
IN	STM32 - GPIOA pin 2
GND	STM32 - GND
NC	NONE
COM	Box Fan
NO	120VAC

4 System Design

4.1 Bill of Material (BOM)

Number of Items	Item	Cost	Link
1	Nucleo STM32L552Ze-Q	$21.02000	https://www.digikey.com/en/products/detail/stmicroelectronics/NUCLEO-L552ZE-Q/11501277
1	GROVE - LASER PM2.5 SENSOR (HM3301)*	$32.90000	https://www.digikey.com/en/products/detail/seeed-technology-co-ltd/101020613/9771839

1	LED CBI 5.08X1.78MM 10SEG G VERT	$1.38000	https://www.digikey.com/en/products/detail/liteon/LTA-1000G/153278
1	RES ARRAY 10 RES 330 OHM 11SIP	$0.53000	https://www.digikey.com/en/products/detail/bourns-inc/4611X-101-331LF/3787988
1	Relay Module(Set of 6)*	$10.99000	https://www.amazon.com/dp/B09G65YFZ6?ref=ppx_yo2ov_dt_b_fed_asin_title
1	20” Box Fan	$23.98	https://www.homedepot.com/p/Lasko-20-in-3-Speed-Box-Fan-3733/308483563
2	MERV-13 20x20x1 Air Filters	Buy 4 or more for $12.49	https://www.homedepot.com/p/HDX-20-in-x-20-in-x-1-in-Premium-Pleated-Furnace-Air-Filter-FPR-10-MERV-13-HDX1P10-012020/315964939

Total Cost of Components: $103.29

These cost values are from the current online prices as of the date of this report. The STM MCU used for this device is overkill and a less expensive version could be used. There is also a good chance of being able to source a less expensive box fan for this purpose by checking local stores and second hand markets.

4.2 Calibration and test procedures

The National Ambient Air Quality Standards (NAAQS) for PM state primary standards as, “public health protection, including protecting the health of "sensitive" populations such as asthmatics, children, and the elderly”[6] should have a yearly average PM level of less than 9.0 µg/m3. While secondary standards are stated as to, “provide public welfare protection, including protection against decreased visibility and damage to animals, crops, vegetation, and buildings.” These should have a PM level less than 15.0 µg/m3. Correlating these standards with the national averages of about 8 2.5PM provides a good knowledge base to set threshold values from. Some environments may aim for cleaner air than the 9.0 or 15 µg/m³ standards.

Based on this information the default threshold values are 10 µg/m3 for the high threshold to turn the system on and the low threshold is 5 µg/m3. Running the sensor to log average values helps tailor thresholds to specific needs. Thresholds are adjusted via preprocessor macros or left at the default value of 10 µg/m³.

5 Conclusion on Product Design Specification

The Automatic Air Analyzer project was a lot of fun and frustration. The two largest challenges I faced were time and finding the right datasheet information. The datasheet on the DigiKey product page doesn’t explain much about how the sensor communicates. This datasheet leads to a Grove seed website that covers the product and links to a GitHub repository with their example Arduino code. This GitHub repository links to the real datasheet for the HM3000&3006 series sensors. This datasheet specifies the device address,

the command code to switch from UART to I2C mode, and the read command and data format. The main confusion came from this datasheet saying the device address was 0x80, while the Grove datasheet stated it as 0x40. I was only confident in trying to communicate with the sensor after downloading and reading through the Arduino code, scanning it for commands and addresses. After some trial and error between wrong addresses and some fixing of misconfigured registers, I was able to get a signal that the device was on the bus.

There are many ways for this project to be improved. One particular change would be to send UART data from the PC to the MCU to set the high and low thresholds. Currently, these can only be configured from preprocessor macros. Another feature that I would like to add to this is some data storage, like flash memory, to keep a history of air quality over a long time. Then, be able to upload this data to the computer to see how and when the sensor is most active. Also, there should be a way to set up the sensor so it can be put in sleep mode. This way, instead of the sensor constantly running and being checked every 5 seconds, it can be set to power up every hour or so, take a reading, and if the air is above threshold, run the fan until the particle count drops and go back to sleep. It would be a much more energy-efficient option for something supposed to run all day. Finally, it would be nice to have all components on a single board and in a nice enclosure.

Appendix A: References

The following table summarizes the documents referenced in this document.

Document Name and Version	Description	Location
[1]EPA Particulate Matter (PM2.5) Trends	Shows Particulate Matter (PM2.5) average levels over national and regional areas from 2010 to 2023	https://www.epa.gov/air-trends/particulate-matter-pm25-trends
[2]Seeed Studio Grove - Laser PM2.5 Sensor (HM3301) Datahseet	Seeed StudioGrove - Laser PM2.5 Sensor (HM3301) Datasheet	https://mm.digikey.com/Volume0/opasdata/d220001/medias/docus/738/Grove_Laser_PM2.5_Sensor_HM3301_Web.pdf
[3]101020613 DigiKey part website	Infrared (IR) Dust Sensor Module DigiKey vendor website	https://www.digikey.com/en/products/detail/seeed-technology-co-ltd/101020613/9771839
[4]Home Depot website for 20” box fan	Product page of 20” box fan from Home Depot	https://www.homedepot.com/p/Lasko-20-in-3-Speed-Box-Fan-3733/308483563
[5]Amazon Website for Relay Module	Amazon webpage of 3V 1 channel relay module	https://www.amazon.com/dp/B09G65YFZ6
[6]NAAQS Table	Table covering national ambient air quality standards for PM	https://www.epa.gov/criteria-air-pollutants/naaqs-table

Appendix B: Key Terms

The following table provides definitions for terms relevant to this document.

Term	Definition
MCU	Microcontroller Unit
PM	Particulate Matter